

CMU · 15-721 | Advanced Database Systems (2020)

15-721 (2021) · 课程资料包 @ShowMeAI



视频
中英双语字幕



课件
一键打包下载



笔记
官方笔记翻译



代码
作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1wv411w7Ko>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-15-721>

内存数据库

并发控制

数据库压缩

哈希连接

超内存数据库

存储模型

调度规划

查询执行

索引

协议

网络

查询编译

查询优化

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程资料包页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击底部菜单栏
称为 AI 内容创作者？回复 [添砖加瓦]

Lecture #16

Carnegie Mellon University

ADVANCED DATABASE SYSTEMS

Vectorization vs. Compilation

@Andy_Pavlo // 15-721 // Spring 2020

OBSERVATION

Vectorization can speed up query performance.

Compilation can speed up query performance.

We have not discussed which approach is better and under what conditions.

Switching an existing DBMS is difficult, so one must make this design decision early.

VECTORWISE – PRECOMPILED PRIMITIVES

Pre-compiles thousands of "primitives" that perform basic operations on typed data.

→ Using simple kernels for each primitive means that they are easier to vectorize.

The DBMS then executes a query plan that invokes these primitives at runtime.

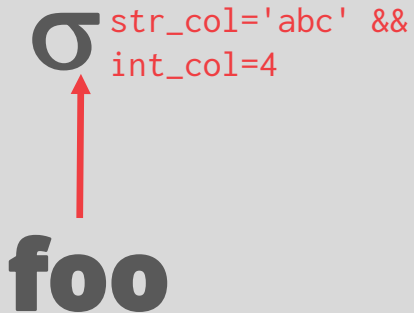
→ Function calls are amortized over multiple tuples.

→ The output of a primitive are the offsets of tuples that



VECTORWISE – PRECOMPILED PRIMITIVES

```
SELECT * FROM foo
WHERE str_col = 'abc'
AND int_col = 4;
```



```
vec<offset> sel_eq_str(vec<string> col, string val) {
    vec<offset> res;
    for (offset i = 0; i < col.size(); i++)
        if (col[i] == val) res.append(i);
    return (res);
}
```

```
vec<offset> sel_eq_int(vec<int> col, int val,
                      vec<offset> positions) {
    vec<offset> res;
    for (offset i : positions)
        if (col[i] == val) res.append(i);
    return (res);
}
```

HYPER – JIT QUERY COMPILATION

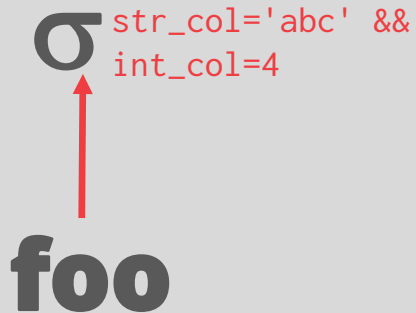
Compile queries in-memory into native code using the LLVM toolkit.

Organizes query processing in a way to keep a tuple in CPU registers for as long as possible.

- Bottom-to-top / push-based query processing model.
- Not vectorizable (as originally described).

HYPER – JIT QUERY COMPILATION

```
SELECT * FROM foo
WHERE str_col = 'abc'
AND int_col = 4;
```



```
vec<offset> sel_eq_row(vec<string> str_col, string val0,
                      vec<int> int_col, int val1) {
    vec<offset> res;
    for (offset i = 0; i < str_col.size(); i++)
        if (str_col[i] == val0 && int_col[i] == val1)
            res.append(i);
    return (res);
}
```

TODAY'S AGENDA

Vectorization vs. Compilation
Relaxed Operator Fusion



VECTORIZATION VS. COMPIRATION

Test-bed system to analyze the trade-offs between vectorized execution and query compilation.

Implemented high-level algorithms the same in each system but varied the implementation details.

→ Example: Hash join algorithm is the same, but the systems use different hash functions (Murmur2 vs. CRC)

IMPLEMENTATIONS

Approach #1: Tectorwise

- Break operations into pre-compiled primitives.
- Must materialize the output of primitives at each step.

Approach #2: Typer

- Push-based processing model with JIT compilation.
- Process a single tuple up entire pipeline without materializing the intermediate results.

TPC-H WORKLOAD

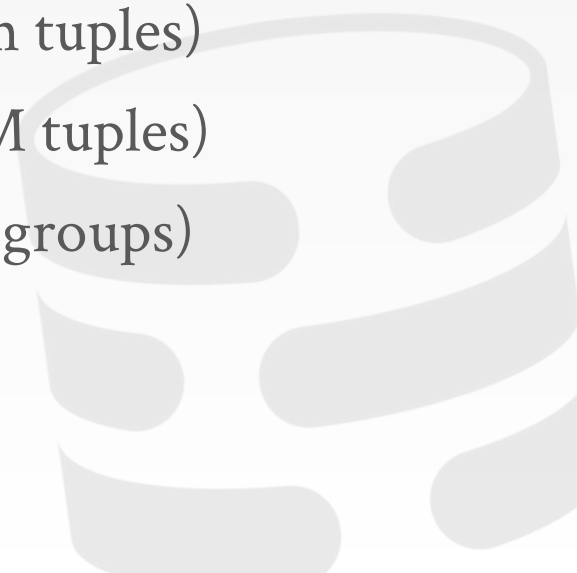
Q1: Fixed-point arithmetic, 4-group aggregation

Q6: Selective filters

Q3: Join (build: 147k tuples / probe: 3.2m tuples)

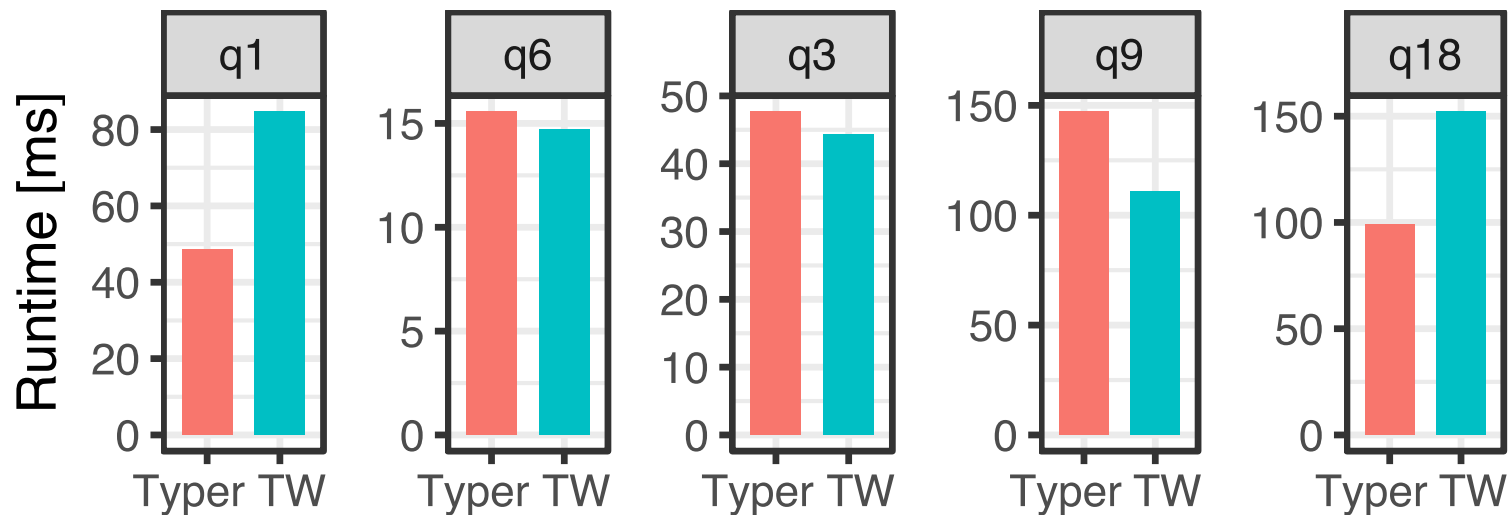
Q9: Join (build: 320k tuples / probe: 1.5M tuples)

Q18: High-cardinality aggregation (1.5m groups)



TPC-H ANALYZED: HIDDEN MESSAGES AND LESSONS LEARNED
FROM AN INFLUENTIAL BENCHMARK
TPCTC 2013

SINGLE-THREADED PERFORMANCE



Source: [Timo Kersten](#)

SINGLE-THREADED PERFORMANCE

		<i>Cycles</i>	<i>IPC</i>	<i>Instr.</i>	<i>L1 Miss</i>	<i>LLC Miss</i>	<i>Bran. Miss</i>
Q1	Typer ★	34	2.0	68	0.6	0.57	0.01
	TW	59	2.8	162	2.0	0.57	0.03
Q6	Typer	11	1.8	20	0.3	0.35	0.06
	TW ★	11	1.4	15	0.2	0.29	0.01
Q3	Typer	25	0.8	21	0.5	0.16	0.27
	TW ★	24	1.8	42	0.9	0.16	0.08
Q9	Typer	74	0.6	42	1.7	0.46	0.34
	TW ★	56	1.3	76	2.1	0.47	0.39
Q18	Typer ★	30	1.6	46	0.8	0.19	0.16
	TW	48	2.1	102	1.9	0.18	0.37

MAIN FINDINGS

Both models are efficient and achieve roughly the same performance.

Data-centric is better for "calculation-heavy" queries with few cache misses.

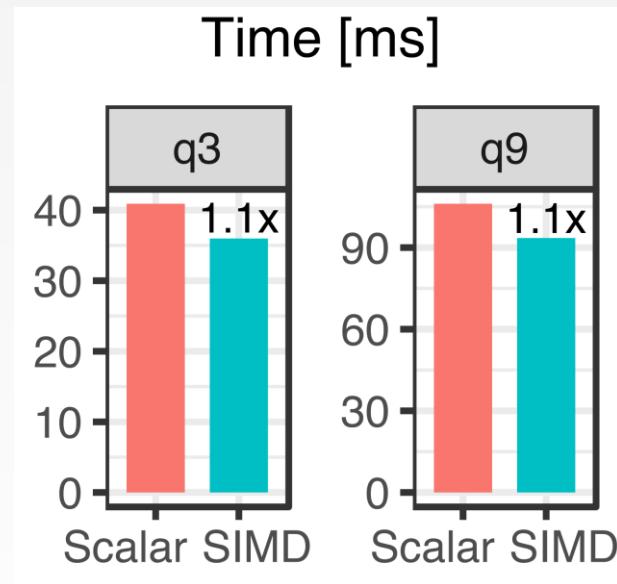
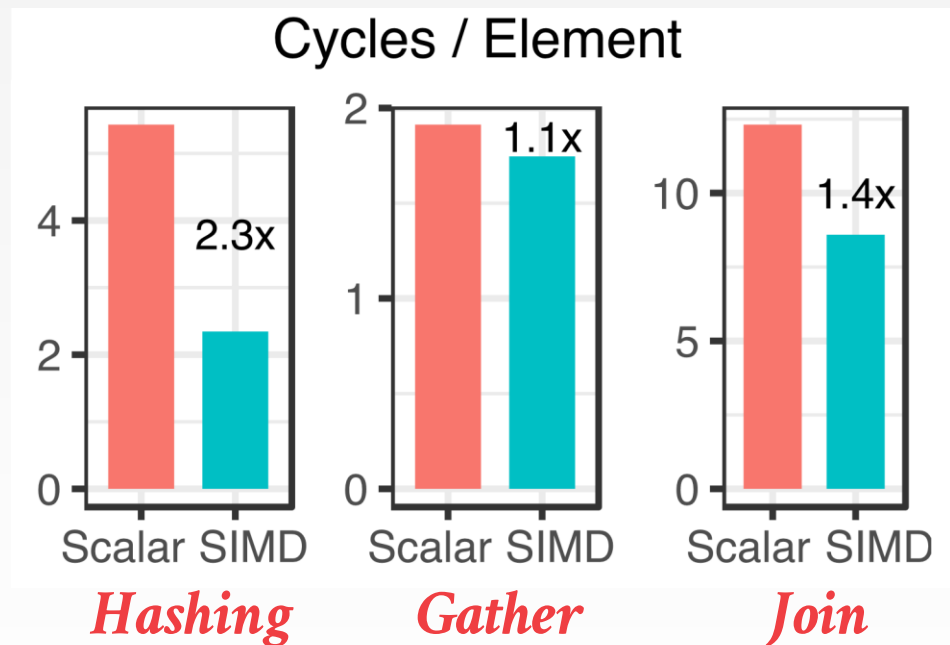
Vectorization is slightly better at hiding cache miss latencies.

SIMD PERFORMANCE

Evaluate vectorized branchless selection and hash probe in Tectorwise.

We use AVX-512 because it includes new instructions to make it easier to implement algorithms using vertical vectorization.

SIMD EVALUATION



Source: [Timo Kersten](#)

AUTO-VECTORIZATION

Measure how well the compiler can automatically vectorize the Vectorwise primitives.

→ Targets: GCC v7.2, Clang v5.0, ICC v18

ICC was able to vectorize the most primitives using AVX-512:

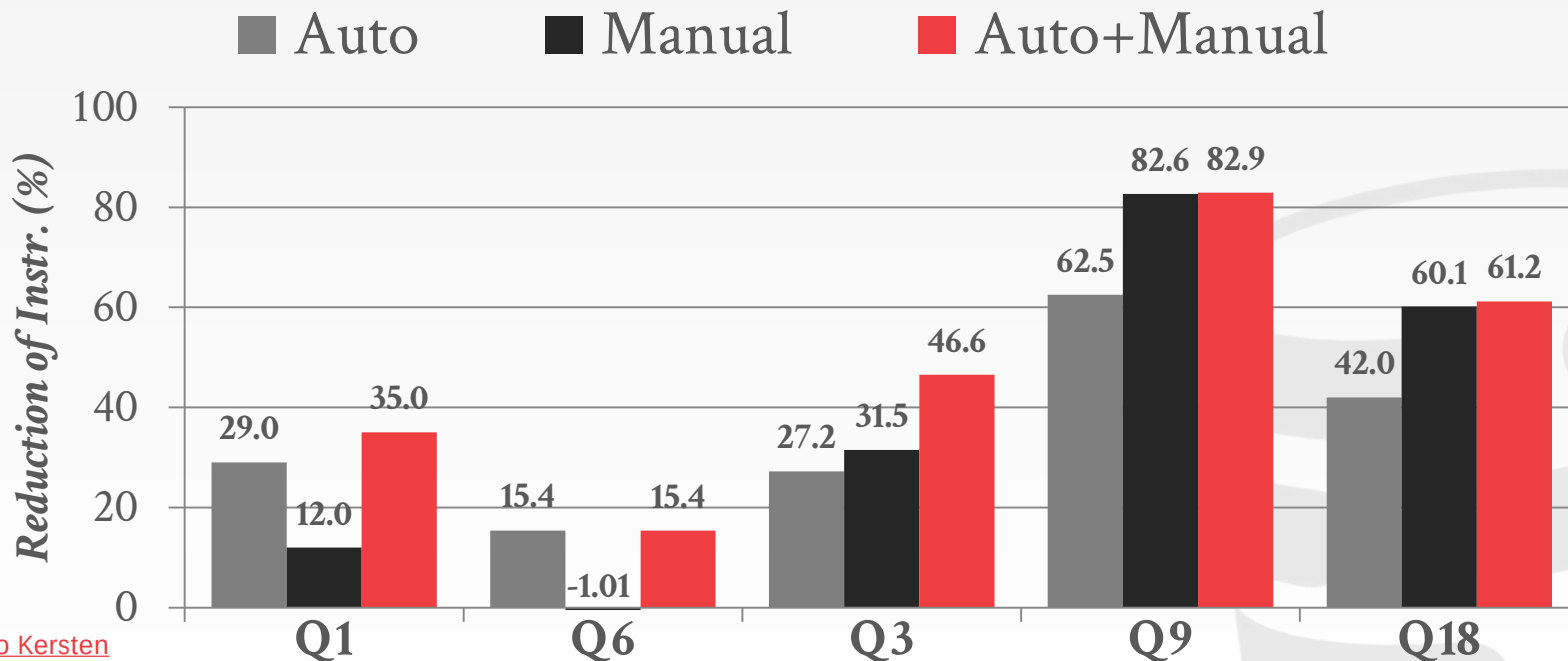
→ Vectorized: Hashing, Selection, Projection

→ Not Vectorized: Hash Table Probing, Aggregation

AUTO-VECTORIZATION

Intel Core i9-7900X (10 cores × 2HT)

Compiler: ICC v18

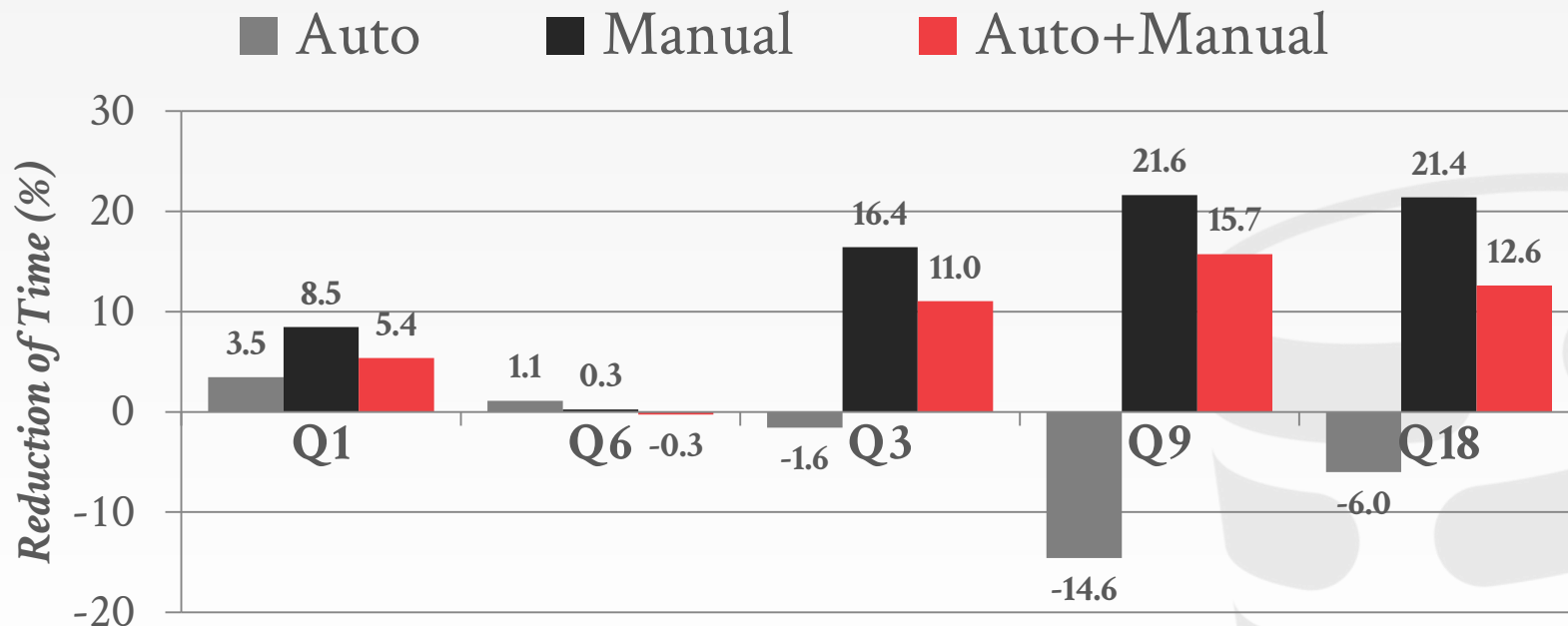


Source: [Timo Kersten](#)

AUTO-VECTORIZATION

Intel Core i9-7900X (10 cores × 2HT)

Compiler: ICC v18



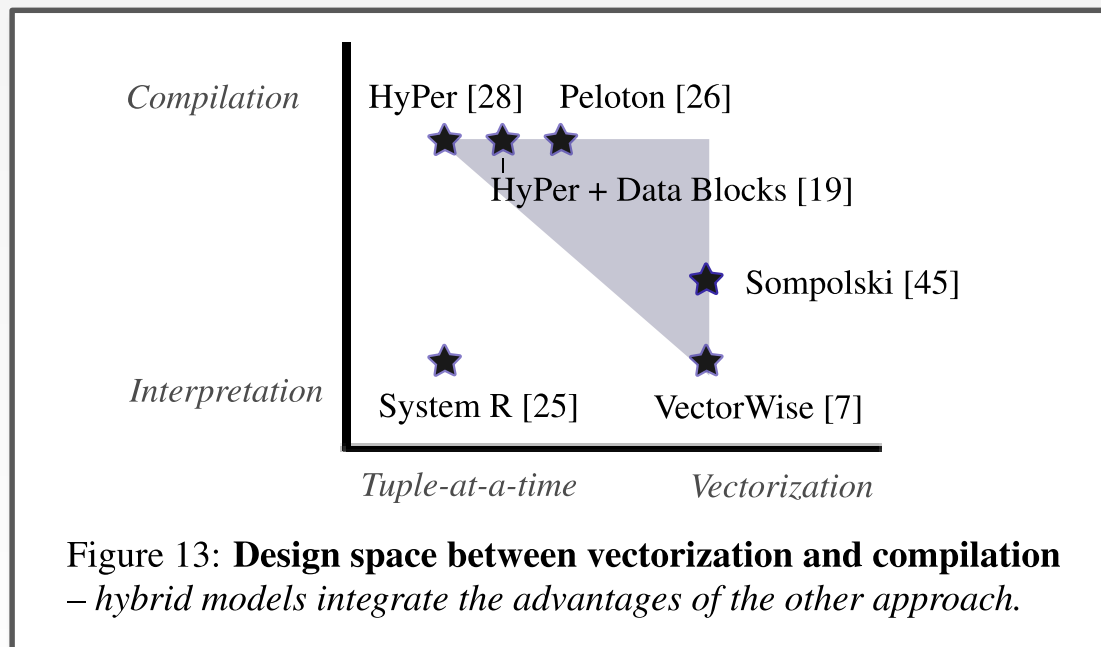
Source: [Timo Kersten](#)

OBSERVATION

The paper (partially) assumes that vectorization and compilation are mutually exclusive.

HyPer fuses operators together so that they work on a single tuple a time to maximize CPU register reuse and minimize cache misses.

VECTORIZATION VS. COMPILATION



Source: [Timo Kersten](#)

PIPELINE PERSPECTIVE

Each pipeline **fuses** operators together into loop

Each pipeline is a **tuple-at-a-time** process



PIPELINE PERSPECTIVE

Each pipeline **fuses** operators together into loop

Each pipeline is a **tuple-at-a-time** process

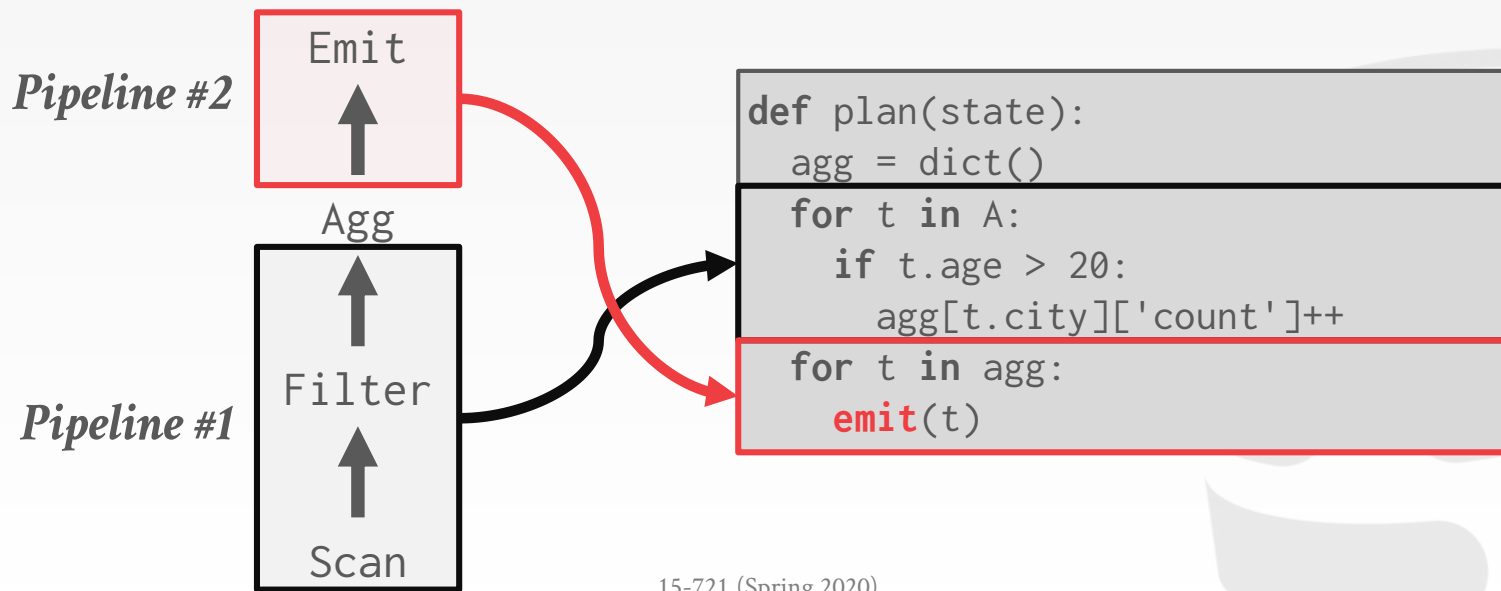


```
def plan(state):  
    agg = dict()  
    for t in A:  
        if t.age > 20:  
            agg[t.city]['count']++  
    for t in agg:  
        emit(t)
```

PIPELINE PERSPECTIVE

Each pipeline **fuses** operators together into loop

Each pipeline is a **tuple-at-a-time** process



FUSION PROBLEMS

Fusion inhibits some optimizations:

- Unable to look ahead in tuple stream.
- Unable to overlap computation and memory access.

```
def plan(state):  
    agg = dict()  
Scan → for t in A:  
Filter →     if t.age > 20:  
Agg →         agg[t.city]['count']++  
    for t in agg:  
        emit(t)
```

FUSION PROBLEMS

Fusion inhibits some optimizations:

- Unable to look ahead in tuple stream.
- Unable to overlap computation and memory access.



Cannot SIMD

Scan

Filter

Agg

```
def plan(state):  
    agg = dict()  
    for t in A:  
        if t.age > 20:  
            agg[t.city]['count']++  
    for t in agg:  
        emit(t)
```

FUSION PROBLEMS

Fusion inhibits some optimizations:

- Unable to look ahead in tuple stream.
- Unable to overlap computation and memory access.



Cannot SIMD



Cannot Prefetch

Scan

Filter

Agg

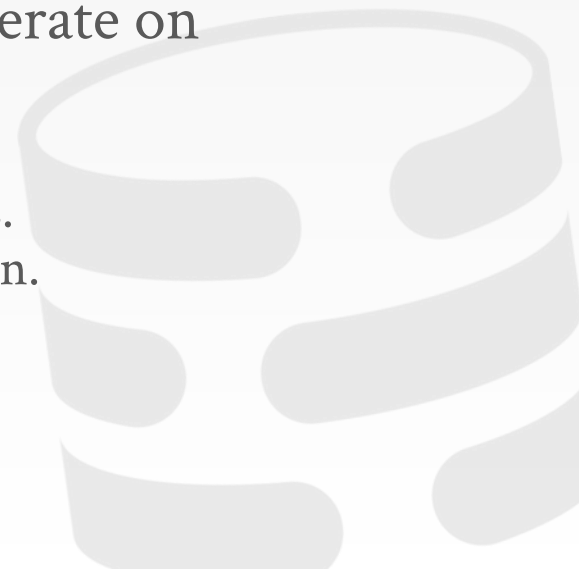
```
def plan(state):  
    agg = dict()  
    for t in A:  
        if t.age > 20:  
            agg[t.city]['count']++  
    for t in agg:  
        emit(t)
```

RELAXED OPERATOR FUSION

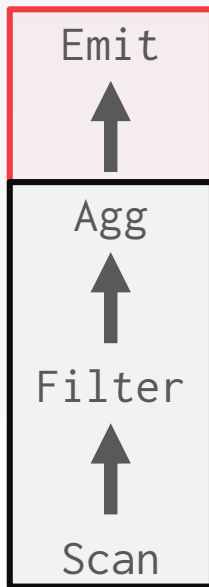
Vectorized processing model designed for query compilation execution engines.

Decompose pipelines into stages that operate on vectors of tuples.

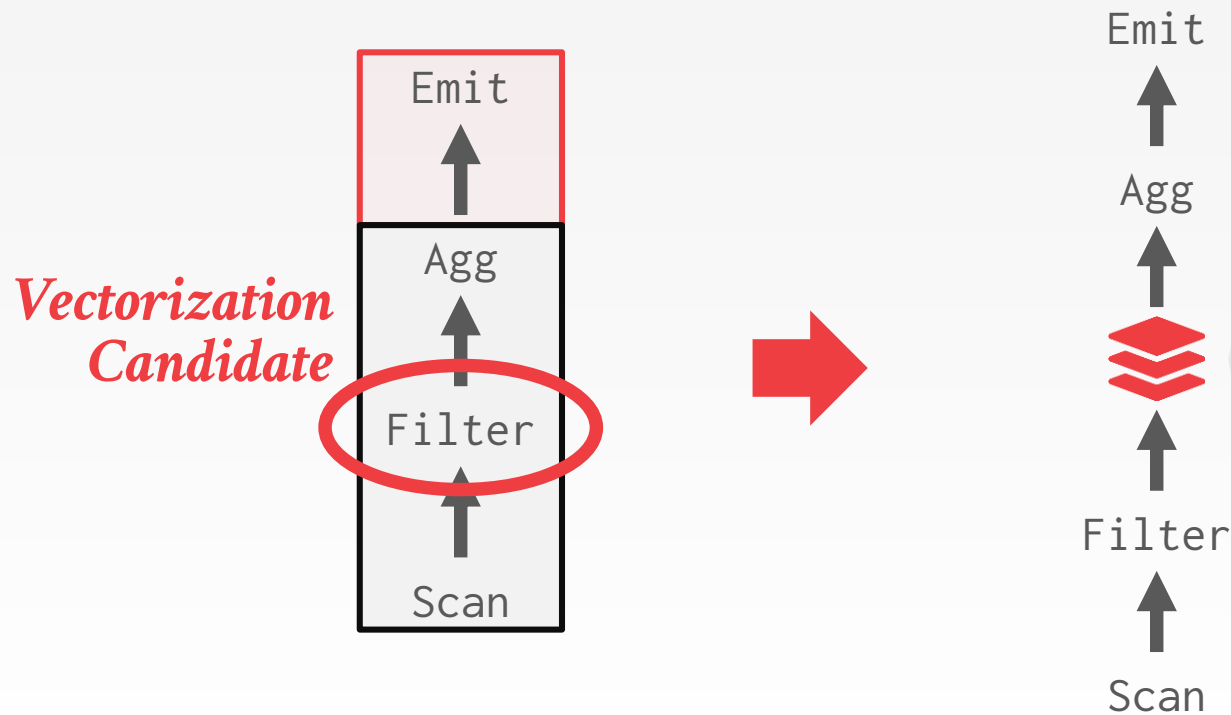
- Each stage may contain multiple operators.
- Communicate through cache-resident buffers.
- Stages are granularity of vectorization + fusion.



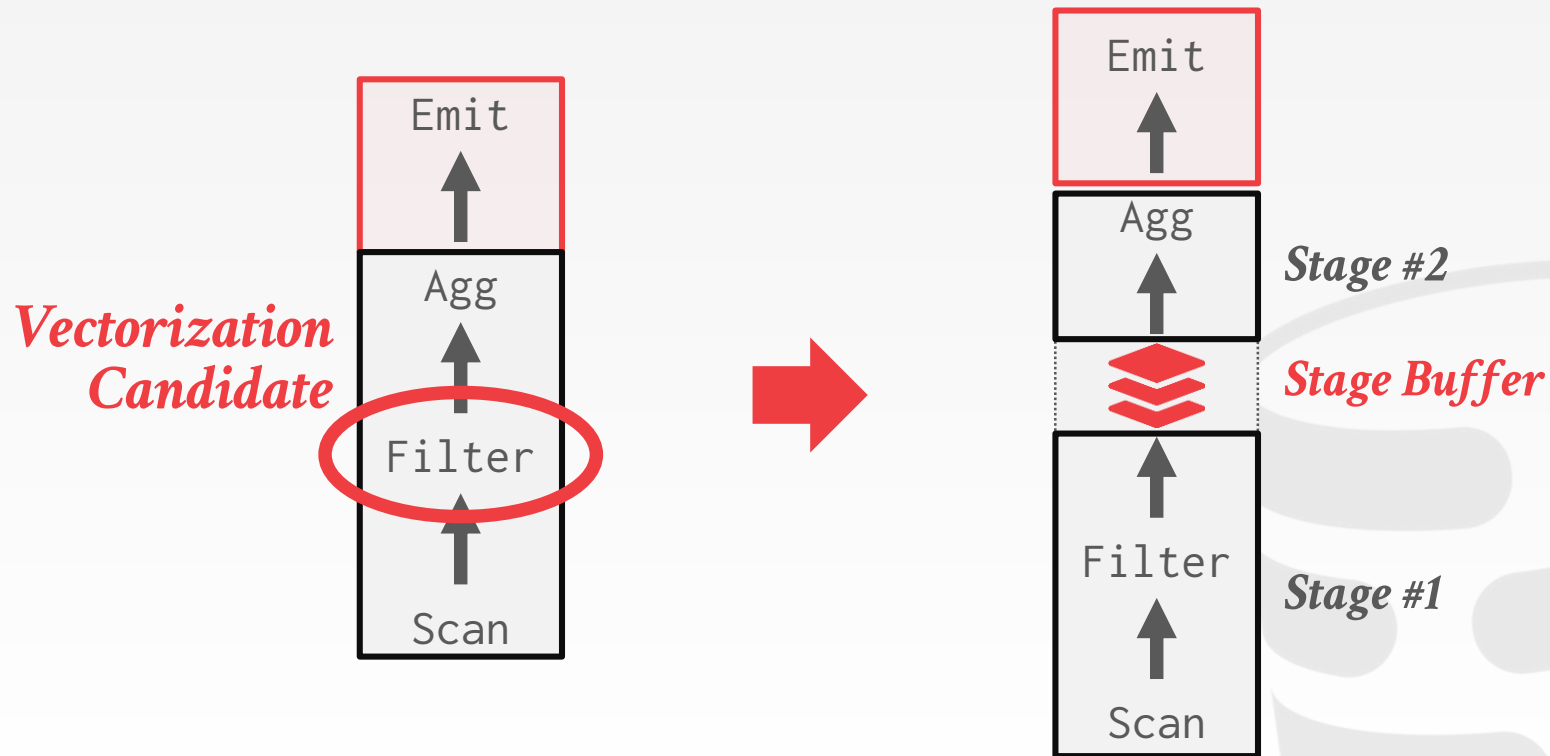
ROF EXAMPLE



ROF EXAMPLE

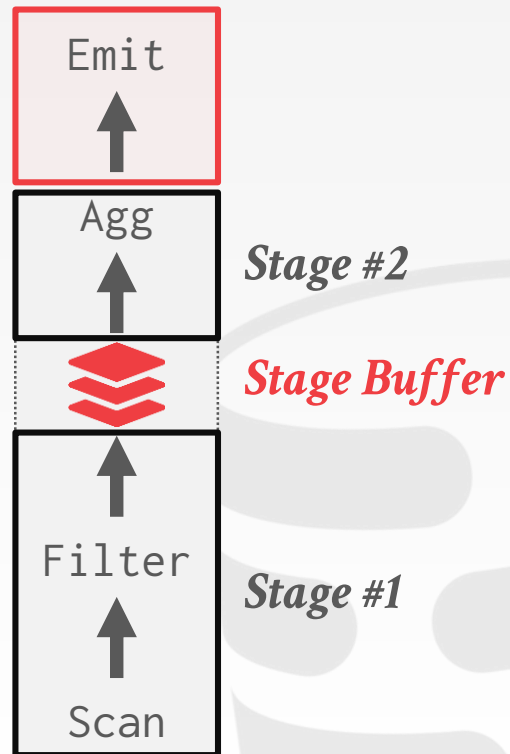


ROF EXAMPLE



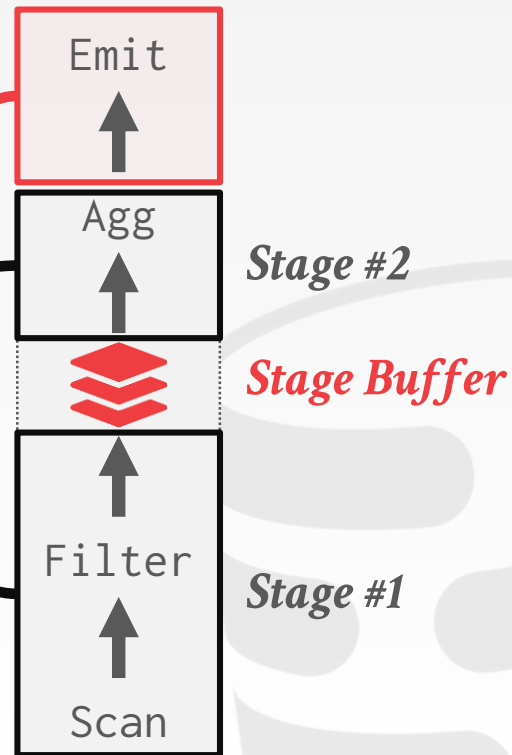
ROF EXAMPLE

```
def plan(state):  
    agg = dict()  
    for t in A step 1024:  
        out = simd_cmp_gt(t, 20, 1024)  
        for ft in out:  
            agg[ft.city]['count']++  
    for t in agg:  
        emit(t)
```



ROF EXAMPLE

```
def plan(state):  
    agg = dict()  
    for t in A step 1024:  
        out = simd_cmp_gt(t, 20, 1024)  
        for ft in out:  
            agg[ft.city]['count']++  
    for t in agg:  
        emit(t)
```



ROF SOFTWARE PREFETCHING

The DBMS can tell the CPU to grab the next vector while it works on the current batch.

- Prefetch-enabled operators define start of new stage.
- Hides the cache miss latency.

Any prefetching technique is suitable

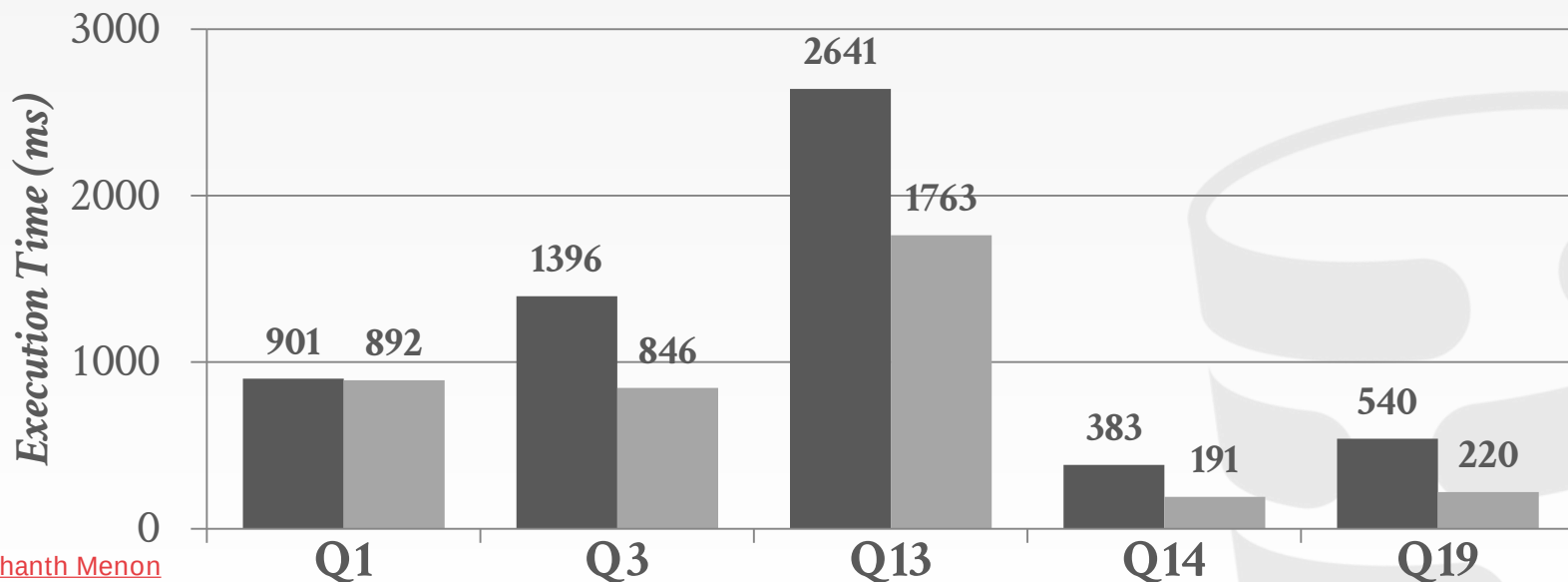
- Group prefetching, software pipelining, AMAC.
- Group prefetching works and is simple to implement.

ROF EVALUATION

Dual Socket Intel Xeon E5-2630v4 @ 2.20GHz
TPC-H 10 GB Database

■ LLVM

■ LLVM + ROF



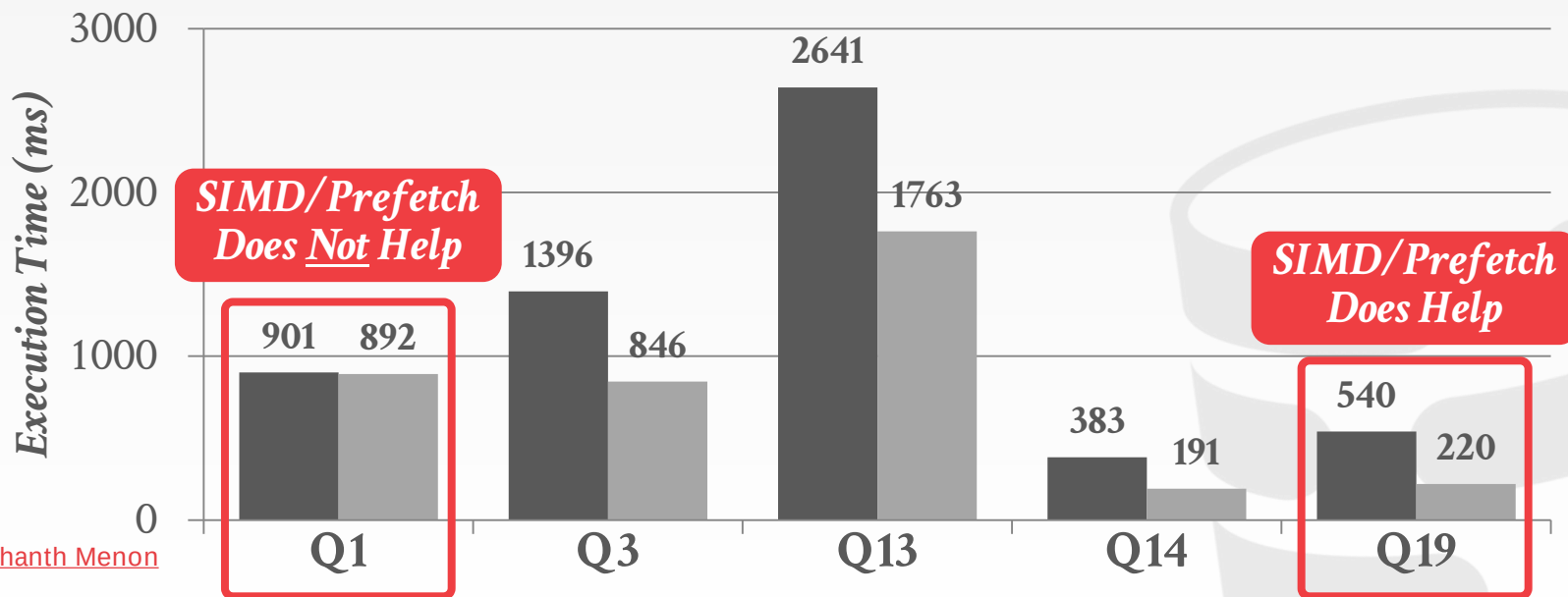
Source: [Prashanth Menon](#)

ROF EVALUATION

Dual Socket Intel Xeon E5-2630v4 @ 2.20GHz
TPC-H 10 GB Database

■ LLVM

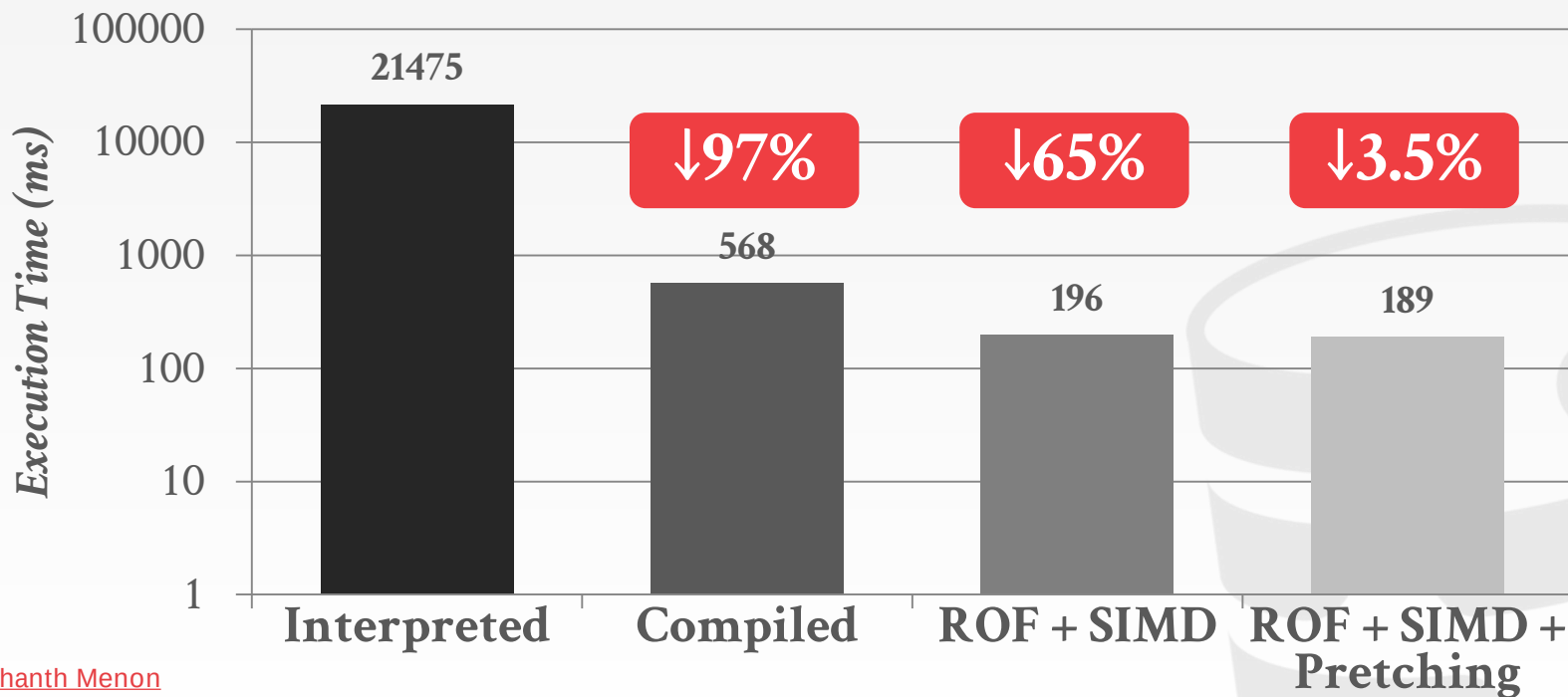
■ LLVM + ROF



Source: [Prashanth Menon](#)

ROF EVALUATION – TPC-H Q19

*Dual Socket Intel Xeon E5-2630v4 @ 2.20GHz
TPC-H 10 GB Database*



Source: [Prashanth Menon](#)

PARTING THOUGHTS

No major performance difference between the Vectorwise and HyPer approaches for all queries.

ROF combines vectorization and compilation into a hybrid query processing model.

→ Trades off additional instructions for reduced CPI

NEXT CLASS

Hash Join Implementations



CMU · 15-721 | Advanced Database Systems (2020)

15-721 (2021) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1wv411w7Ko>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-15-721>

内存数据库

并发控制

数据库压缩

哈希连接

超内存数据库

存储模型

调度规划

查询执行

索引

协议

网络

查询编译

查询优化

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程资料包页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击底部菜单栏
称为 AI 内容创作者？回复 [添砖加瓦]